MPOO 2027-1 · GRUPO 06

Parcial 1 · correcciones

Versión D del examen del 7 de octubre. Para cada pregunta vienen el enunciado, tu respuesta tal como la enviaste en el portal y su corrección. La calificación no va aquí: la ves con el profesor.

Antes de empezar. Este archivo lo subió el profesor a tu rama. Trae el cambio a tu computadora antes de tu siguiente push, o tu push va a fallar:

```
git checkout entregas_romero_rodolfo
git pull --no-rebase
```

Qué tienes que hacer. En hojas, **a mano**, copia cada pregunta y escribe debajo tu respuesta corregida, usando la corrección que viene aquí. Donde la corrección ya te da la salida o el código, no basta con copiarlo: escribe **por qué** sale así, con la tabla de seguimiento de las variables, la causa del error o lo que devuelve tu método en cada renglón de la tabla. Debajo de esas preguntas viene qué escribir. Escanea las hojas o tómales foto, júntalas en un PDF y súbelo a esta misma carpeta, `entregas/romero_rodolfo/examen/p1/`, con tu push de siempre.

Pregunta 0.1 · Preguntas abiertas

Pregunta

Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de código
¿Qué hace new al crear un objeto? Explica qué recibe la variable a la que asignas esa expresión.

Tu respuesta

```
Dentro de la funciòn New, nos permite poder crear un nuevo objeto. Este al momento de ser
creado se le debe asignar un nuevo nombre junto con el tipo de Dato que se va a manejar.
```

Corrección. Reconoce que new crea un nuevo objeto. Le falta decir que new ejecuta el constructor para inicializar la instancia y que la variable recibe una referencia al objeto creado, no un nombre ni un tipo.

Pregunta 0.2 · Preguntas abiertas

Pregunta

¿Qué significa void en la cabecera de un método? Explica si ese método puede modificar un atributo.

Tu respuesta

```
Void es el lo que se estará regresando dentro del programa al momento de que este se finalice,
si puede modificar un atributo al momento de cambiar el parametro que se estarà usando.
```

Corrección. Acierta en que el método puede modificar un atributo. Pero void no es lo que se regresa al terminar: indica que el método no devuelve ningún valor a quien lo llama, aunque sí puede producir efectos como cambiar el estado.

Pregunta 0.3 · Preguntas abiertas

Pregunta

¿Para qué sirve this? Explica cómo distingue un atributo de un parámetro que tiene el mismo nombre.

Tu respuesta

```
El This a comparaciòn de los demàs es el que permite traer o "invocar" a otro parametro
distinguido por la letra mayuscula, como si fuer This.Compu.compu
```

Corrección. Es incorrecta: this no invoca otro parámetro ni tiene que ver con las mayúsculas. this es una referencia al objeto actual; en this.temperatura = temperatura, la izquierda es el atributo y la derecha el parámetro del mismo nombre.

Pregunta 0.4 · Preguntas abiertas

Pregunta

Java siempre pasa por valor. ¿Qué recibe un método cuando le pasas un int y cuando le pasas un arreglo?

Tu respuesta

```
Uno se volvería directamente un valor fijo del arreglo, mientras que en el arreglo formaría la
cantidad de espacios para un arreglo.
```

Corrección. La respuesta no explica qué recibe el método. En los dos casos recibe una copia del valor de la variable: el número si es int y la referencia si es un arreglo, así que modificar un elemento sí afecta al arreglo del llamador, pero reasignar el parámetro no.

Pregunta 0.5 · Preguntas abiertas

Pregunta

¿En qué se distinguen JDK y JRE? Explica cuál necesitas para compilar un programa y por qué.

Tu respuesta

```
JDK es el funcionamiento del programa, revisa que todos los datos son legales dentro del
sistema, sin embaro el JRE es el que se encarga de hacer el compilado de la consola, es como
un intermediario dentro del programa para que este se pueda ejecutar y mostrar al usuario.
```

Corrección. Conserva una idea válida: el JRE sirve para ejecutar el programa. Pero invierte los papeles: el JRE no compila; es el JDK el que trae las herramientas de desarrollo, como javac, y por eso se necesita para compilar.

R1 · ¿Qué imprime?

Pregunta

Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido.

```
Integer base = 5;
Integer copia = base;
int[] aumentos = {1, 2, 3};
```

```
for (int aumento : aumentos) {
    base = base + aumento;
    System.out.println(base / 2);
}
String etiqueta = "D" + copia + 1;
System.out.println(etiqueta);
System.out.println((double) (base / 2));
```

Tu respuesta

```
Etiqueta = D51
Double = 5.0
```

Corrección. Los valores D51 y 5.0 son correctos, pero tienen prefijos ('Etiqueta = ', 'Double = ') que no forman parte de la salida exacta. Faltan las tres líneas del ciclo: base pasa por 6, 8 y 11, y se imprime 3, 4 y 5. La salida completa es 3 / 4 / 5 / D51 / 5.0.

Qué escribir a mano. La salida ya la tienes en la corrección. Escribe por qué sale así: una tabla de seguimiento con el valor de cada variable en cada vuelta y en cada println.

R2 a) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
if (true) { int voltaje = 12; }
System.out.println(voltaje);
```

Tu respuesta

Clasificación

```
No compila
```

Causa o salida

```
No tiene lógica el código.
```

Corrección. La clasificación 'No compila' es correcta. La causa no es concreta: voltaje se declara dentro del bloque del if y no existe fuera de él, aunque la condición sea true.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 b) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
StringBuilder ruta = new StringBuilder("Q");
ruta.append(2 + 3);
```

```
System.out.println(ruta);
```

Tu respuesta

Clasificación

```
Funciona
```

Causa o salida

```
Q
```

> **Corrección.** La clasificación 'Funciona' es correcta, pero la salida es Q5, no Q. Primero se calcula 2 + 3 = 5 y luego append lo agrega al StringBuilder.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 c) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
int[] canales = null;
System.out.println(canales.length);
```

Tu respuesta

Clasificación

```
Compila pero truena
```

Causa o salida

```
El .length genera el error de lógica en el código
```

> **Corrección.** La clasificación es correcta y señala bien el punto donde falla (.length). Falta la causa: canales vale null, así que no hay arreglo cuyo length consultar y se lanza NullPointerException. Tampoco es un error de lógica, sino de ejecución.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R3 · Modelado: base, reto y justificación

Pregunta

Cada termostato guarda etiqueta fija y temperatura. El rango común del edificio es de 16 a 28 grados. Constructor y ajustar(int temperatura) reciben valores válidos; ajustar asigna. Añade una consulta de temperatura.

Base. Declara al menos dos atributos de instancia con tipo, uno static y una constante final (¿también static?). Completa constructor y dos métodos: cambio y consulta.

Reto. ¿Cómo declaras los dos extremos del rango para que sean comunes y no cambien?

Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

Tu respuesta

Clase

```
static int promedio(int[] Temper)
{
if (Temper>=16 && <=28) {
    System.out.println(Temper);
}
int selec=1, int cons=2;
for(int i=0; i!=0 && i<3);
Scanner T = new Scanner;
*Se solicita dato con scann*
if (i==1)
{
   System.out.println("La temperatura es" +Temp);
}
else if (i==2)
{
   System.in("Dame la nueva temperatura" i);
}
else (i<1||i>3)
{
   System.out.println("Error");
}
}
```

Justificaciones

```
Está en un ciclo para que esté en funcion continua
Puse el rango de temperatura para tenerlo siempre guardado
El for se usa de menú aun siendo ciclo para estar solicitando la opción
Los if imprimen las opciones deseadas o error en su defecto.
Usé el código de arriba como base.
```

Corrección. Lo que escribió es un método static con un menú, Scanner y estructuras de control, no la clase Termostato. No hay atributos de instancia (etiqueta, temperatura), ni atributo static, ni constante final, ni constructor con this, ni los métodos ajustar/consultar. Para el reto se esperaban los extremos como static final int MINIMO = 16 y MAXIMO = 28, no números literales dentro de un if.

R4 · Código con repetición

Pregunta

Devuelve el promedio entero de los valores pares de pulsos usando for-each. Si no hay pares, devuelve 0. El arreglo puede estar vacío, no es null y admite negativos. La suma cabe en int. No modifiques el arreglo.

```
static int promedioPares(int[] pulsos)
```

Ejemplos (argumentos → retorno):

{3, 8, 2, 5} → 5 · {1, 3, 5} → 0 · {-4, 3, 6} → 1

Escribe el método completo. No imprimas ni leas del teclado.

Tu respuesta

```
static int promedioPares(int[] pulsos)
{
  int[] pulso = {3, 8, -4};
    for (int promedio : pulsos)
    {
        pulso = Pares + pulso;
        System.out.println(pulso % 2);
    }
  System.out.println((double) (pulso%2));

}
```

Corrección. La cabecera es correcta y usa un for-each sobre pulsos. Sin embargo, no filtra pares (solo imprime pulso % 2), no lleva suma ni contador válidos (el intento 'pulso = Pares + pulso' opera sobre un arreglo y una variable no declarada), no maneja el caso sin pares, no tiene return e imprime, cuando el enunciado lo prohíbe. Lo esperado era acumular suma y cantidad de los valores con valor % 2 == 0 y devolver 0 si cantidad == 0, o suma / cantidad si no.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.

R5 · Código con selección

Pregunta

Clasifica la temperatura entera de una cámara de pruebas. Usa if / else-if / else y devuelve el texto de la tabla. Los límites están incluidos. No imprimas ni leas del teclado.

```
static String estadoTermico(int temperatura)
```

Regla o valor recibido	Texto devuelto
Menor que 0 o mayor que 120	"INVALIDA"
0 a 29	"FRIA"
30 a 59	"TEMPLADA"
60 a 89	"CALIENTE"
90 a 120	"MUY CALIENTE"

Tu respuesta

```
static String estadoTermico(int temperatura)
{
    System.out.printl("Temperatura mostrada");
    if (Temperatura >= 0 && Temperatura <=29)
    {
      System.out.printl("Fria");
    }

    else if (Temperatura >= 30 && Temperatura <=59)
    {
      System.out.printl("Templada");
    }
```

```
    else if (Temperatura >= 60 && Temperatura <=89)
    {
      System.out.printl("Caliente");
    }

    else if (Temperatura >= 90 && Temperatura <=29)
    {
      System.out.printl("Muy Caliente");
    }

    else (Temperatura <0 || Temperatura >120)
    {
      System.out.printl("INVALIDA");
    }
}
```

Corrección. La cabecera es correcta y la cadena if/else-if/else tiene bien los rangos 0-29, 30-59 y 60-89, y rechaza los valores fuera de 0-120. Errores: el rango de MUY CALIENTE quedó '>= 90 && <= 29', así que de 90 a 120 se clasifica como INVALIDA. Además imprime en lugar de devolver con return, los textos no coinciden con la tabla ('Fria' en lugar de "FRIA") y hay detalles de sintaxis como printl, Temperatura con mayúscula y un else con condición.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.